Graph Theory Homework

In this assignment you will use the provided code template to implement solutions to each given problem. The solutions you submit must follow the guidelines provided in this document as well as any given in code.

Restrictions

- Template code is provided and must be used.
  - ***Note: The templates are refreshed each semester. Be sure to use the current version.***
- Your code must be compatible with **Python 3.10**.
- Do not add imports for any libraries beyond the Python standard library.
- Do not modify the function name or definition.

Testing

For each problem, one or more base test cases are provided. A properly implemented solution will pass any provided base test cases, but their trivial nature is not intended to ensure the overall correctness of your algorithm.

These are the same test cases that will be used to provide assurance that your submitted solutions run using the autograder. We do not release any other test cases.

In problem instances where more than one solution is optimal, *any* optimal solution will be considered correct, assuming that is the solution your algorithm produces.

The test cases can - and should! - be expanded while developing your solutions.

You can run your test cases by issuing the `python3 -m unittest` command from the directory containing your solution files.

Submission

Submit only - and all of - your solution files (i.e., **cs6515_[problem_name].py**) to the Gradescope assignment on or before the posted due date. Do not submit a zip file or any other files except those matching the naming convention defined.

You may submit your solutions as many times as desired and the most recent submission is what will be graded. Execution of your code may take up to 60 minutes. If not completed before the due date, your previous submission will be used.

Faster and correct solutions are worth more credit.

After your submission completes execution, you will see a score of `- / 20` listed until grading is completed.

Tim's Coffee Dilemma

Tim's favorite coffee shop has closed due to the tough economy. Now he has to find a new coffee shop on his drive to work.

You are given a graph G, where V are intersections and E are **directional** roads. All roads $e \in E$ have a length in units.

You are also provided with a list of intersections C, such that $c \in C$ is an intersection that is close to exactly one coffee shop.

Each $c \in C$ is also provided with a rating of 1-5 indicating it's Yelp review score. For every star, Tim is willing to drive an extra unit of length.

Find the best path from Tim's home (s) to Tim's work (t) such that at least one $c \in C$ is along the path.

The best path is defined as, the shortest path Tim should take, taking into account the rating of the coffee shop Tim stops at. Note that Tim only has to stop at one coffee shop along the way.

For example, a path that takes 25 units and stops at a 4 star coffee shop is **better** than a path that takes 24 units and stops at a 1 star coffee shop.

Design an algorithm to produce a path from Tim's home (s) to Tim's work (t) such that at least one $c \in C$ is along the path. We want the best path as defined. Return the coffee_shop and the path taken.

Notes:

- The best coffee shop should be returned as a Node
- Tim's path should be returned as an ordered list of the starting Node to the ending Node

Provide your code for implementing this algorithm in **cs6515_tim_coffee.py**